

GitFlow: la estrategia que organiza el trabajo de un equipo de desarrollo

Imagina que un equipo de desarrolladores está construyendo una aplicación para gestionar una clínica. Mientras un integrante desarrolla el módulo de pacientes, otro trabaja en las citas médicas y otro corrige errores encontrados por los usuarios. Si todos modificaran el mismo código al mismo tiempo sin organización, sería muy fácil sobrescribir cambios, generar conflictos o incluso dañar una versión que ya funciona.

GitFlow surge precisamente para evitar ese problema. Es una metodología de trabajo basada en **Git** que organiza el desarrollo mediante diferentes ramas (*branches*), donde cada una tiene una responsabilidad específica. De esta manera, el equipo puede trabajar de forma simultánea, segura y ordenada.

En otras palabras, **GitFlow es una estrategia para administrar el ciclo de vida de un proyecto utilizando Git**, permitiendo desarrollar nuevas funcionalidades, corregir errores y preparar versiones sin afectar la estabilidad del sistema.

¿Cómo funciona GitFlow?

GitFlow divide el proyecto en varias ramas principales:

- **main**: contiene la versión estable del sistema, es decir, la que está lista para ser utilizada por los usuarios.
- **develop**: es la rama donde se integran todas las nuevas funcionalidades antes de convertirse en una versión oficial.
- **feature/**: se crea para desarrollar una nueva característica sin afectar el trabajo de los demás.
- **release/**: prepara una nueva versión para producción realizando pruebas y ajustes finales.
- **hotfix/**: permite corregir errores críticos directamente sobre la versión que está en producción.

Ejemplo práctico

Supongamos que un equipo desarrolla un sistema para una universidad.

1. La versión actual del sistema funciona correctamente y está almacenada en la rama **main**.
2. El equipo quiere agregar un módulo para registrar materias. Para ello crea una rama llamada:

```
feature/registro-materias
```

3. Mientras tanto, otro desarrollador crea otra rama para implementar el módulo de profesores:

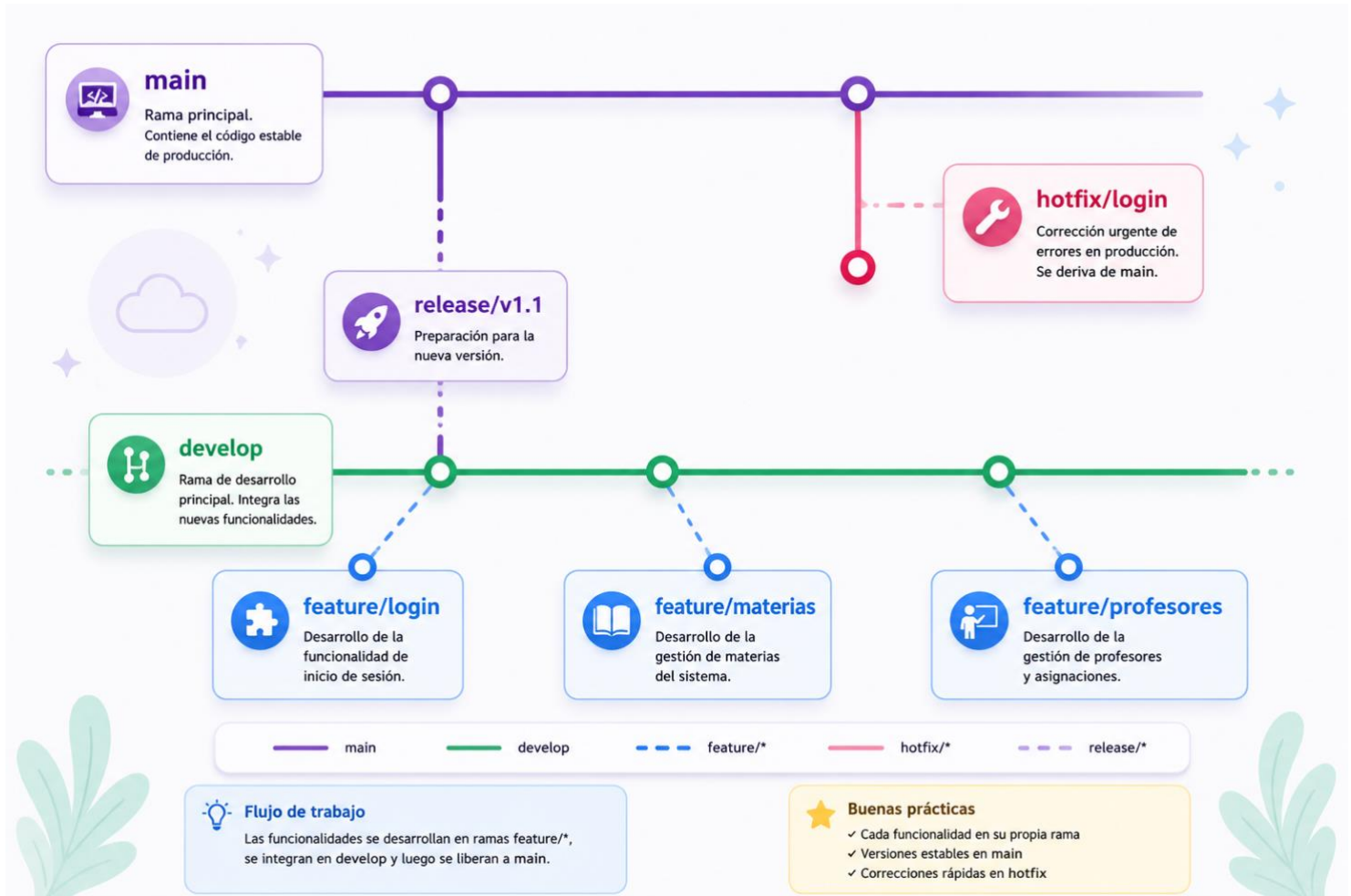
```
feature/modulo-profesores
```

4. Ambos trabajan de manera independiente sin interferir entre sí.
5. Cuando terminan sus tareas, sus cambios se integran en la rama **develop**, donde todo el equipo verifica que el sistema continúe funcionando correctamente.
6. Una vez realizadas las pruebas, se crea una rama **release** para preparar la nueva versión.
7. Finalmente, la versión se publica en la rama **main**.

Si después de publicar aparece un error crítico, por ejemplo que los estudiantes no puedan iniciar sesión, no es necesario esperar al siguiente desarrollo. Se crea una rama:

hotfix/error-login

Se corrige el problema, se prueba y se integra nuevamente en **main** y **develop**.



¿Cuáles son las ventajas de GitFlow?

- Mantiene una versión estable del proyecto en todo momento.
- Permite que varios desarrolladores trabajen simultáneamente sin conflictos constantes.
- Facilita la corrección de errores críticos mediante ramas **hotfix**.
- Organiza el desarrollo de nuevas funcionalidades de forma independiente.
- Hace más sencillo preparar versiones antes de publicarlas.
- Es ideal para proyectos empresariales donde intervienen varios desarrolladores.

En conclusión

GitFlow no es un programa ni una herramienta adicional; es una **metodología de organización del trabajo con Git**. Gracias a esta estrategia, cada cambio tiene un lugar específico dentro del proyecto, lo que mejora la colaboración, reduce errores y garantiza que siempre exista una versión estable del software lista para ser entregada a los usuarios.

En proyectos profesionales, utilizar GitFlow es como construir un edificio siguiendo un plano: cada persona sabe qué debe hacer, dónde trabajar y cuándo integrar su trabajo con el del resto del equipo. El resultado es un desarrollo más ordenado, eficiente y de mayor calidad.

1. Inicializar Git en el proyecto

```
</> Bash
```

```
git init
```

2. Verificar el estado del proyecto

```
</> Bash
```

```
git status
```

3. Agregar todos los archivos

```
</> Bash
```

```
git add .
```

O agregar un archivo específico:

```
</> Bash
```

```
git add nombreArchivo.java
```

O agregar un archivo específico:

⟨⟩ Bash

```
git add nombreArchivo.java
```

4. Crear un commit

⟨⟩ Bash

```
git commit -m "Se crea el proyecto base"
```

5. Conectarse con GitHub

⟨⟩ Bash

```
git remote add origin https://github.com/usuario/repositorio.git
```

Verificar el repositorio remoto:

⟨⟩ Bash

```
git remote -v
```

6. Enviar el proyecto a GitHub

Primera vez:

❏ Bash

```
git branch -M main
git push -u origin main
```

Las siguientes veces:

❏ Bash

```
git push
```

Comandos de GitFlow

Crear la rama develop

❏ Bash

```
git checkout -b develop
```

Subirla a GitHub

❏ Bash

```
git push -u origin develop
```

Crear una rama Feature

Ejemplo:

⟨/⟩ Bash

```
git checkout develop  
git checkout -b feature/login
```

O

⟨/⟩ Bash

```
git switch develop  
git switch -c feature/login
```

Guardar cambios

⟨/⟩ Bash

```
git add .  
git commit -m "Se implementa el módulo de login"
```

Subir la Feature

⟨/⟩ Bash

```
git push -u origin feature/login
```

Volver a develop

⌞/⌟ Bash

```
git checkout develop
```

Unir la Feature con develop

⌞/⌟ Bash

```
git merge feature/login
```

Eliminar la rama Feature

⌞/⌟ Bash

```
git branch -d feature/login
```

Crear una Release

⟨⟩ Bash

```
git checkout develop
git checkout -b release/v1.0
```

Crear un Hotfix

⟨⟩ Bash

```
git checkout main
git checkout -b hotfix/error-login
```

Flujo completo

⟨⟩ Bash

```
git init
git add .
git commit -m "Proyecto inicial"
git branch -M main
git remote add origin https://github.com/usuario/proyecto.git
git push -u origin main
git checkout -b develop
git push -u origin develop
git checkout -b feature/login
git add .
git commit -m "Se crea el módulo de login"
git push -u origin feature/login
git checkout develop
git merge feature/login
git push
git branch -d feature/login
```


Resumen de los comandos más importantes

Comando	Función
git init	Inicializa Git en el proyecto.
git status	Muestra el estado de los archivos.
git add .	Agrega todos los cambios al área de preparación.
git commit -m "mensaje"	Guarda una versión del proyecto.
git branch	Lista las ramas existentes.
git checkout -b nombre-rama	Crea y cambia a una nueva rama.
git switch -c nombre-rama	Crea y cambia a una nueva rama (comando moderno).
git checkout nombre-rama	Cambia de rama.
git switch nombre-rama	Cambia de rama (comando moderno).
git merge nombre-rama	Fusiona una rama con la actual.
git branch -d nombre-rama	Elimina una rama local.
git push	Envía los cambios al repositorio remoto.
git pull	Descarga e integra los cambios del repositorio remoto.

Ejemplo de ramas en GitFlow



Este es el flujo más utilizado en equipos de desarrollo porque permite trabajar en nuevas funcionalidades, corregir errores y preparar versiones de forma organizada sin afectar la versión estable del proyecto.